BookSwap - Technical Architecture

BookSwap - Technical Architecture

1. System Architecture

```
┌──────────────
```

```
│   │   Service     │   │   Service     │   │   Service     │   │   Service
│     │
│   │  (OTP/JWT)    │   │  (CRUD)       │   │  (Search)     │   │ (Push/WA)
│     │
│   └───────────────┘   └───────────────┘   └───────────────┘
└───────────────┘     │
│   ┌───────────────┐   ┌───────────────┐   ┌───────────────┐
│
│   │    User       │   │   Exchange    │   │    Media      │
│
│   │   Service     │   │   Service     │   │   Service     │
│
│   │  (Profile)    │   │  (Workflow)   │   │  (Images)     │
│
│   └───────────────┘   └───────────────┘   └───────────────┘
│
└──────────────────────────────────────────────────────────────────────────────
                                  │
                  ┌───────────────┼───────────────┬───────────────┐
                  ▼               ▼               ▼               ▼
┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│  PostgreSQL  │  │    Redis     │  │Elasticsearch│  │     S3       │
│  (Primary)   │  │   (Cache)    │  │  (Search)    │  │  (Images)    │
└──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘

┌──────────────────────────────────────────────────────────────────────────────
│                          EXTERNAL SERVICES
│
│   ┌───────────────┐   ┌───────────────┐   ┌───────────────┐
│
│   │   WhatsApp    │   │  Firebase     │   │   SendGrid    │
│
│   │  Business     │   │    FCM        │   │   (Email)     │
│
│   │    API        │   │   (Push)      │   │               │
│
│   └───────────────┘   └───────────────┘   └───────────────┘
│
└──────────────────────────────────────────────────────────────────────────────
```

---

2. Tech Stack Recommendation

Option A: Cost-Effective (Recommended for MVP)

Layer	Technology	Reason
Mobile	React Native	Cross-platform, JS ecosystem
Backend	Node.js + Express	Fast development, easy hiring
Database	PostgreSQL	Reliable, free, good for relational data

Layer	Technology	Reason
Cache	Redis	Sessions, OTP storage
Search	PostgreSQL Full-Text	Good enough for MVP
Storage	AWS S3 / Cloudinary	Image hosting
Push	Firebase FCM	Free tier generous
WhatsApp	WhatsApp Business API	Via provider like Gupshup/Twilio
Hosting	Railway / Render / AWS	Start small, scale as needed

Estimated Monthly Cost (MVP): ₹5,000-10,000/month

Option B: Scale-Ready

Layer	Technology
Mobile	Flutter
Backend	NestJS (TypeScript)
Database	PostgreSQL + Read Replicas
Search	Elasticsearch
Queue	Redis + BullMQ
Hosting	AWS ECS / Kubernetes

---

3. Database Schema

3.1 Core Tables

```sql
-- Users
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    phone VARCHAR(15) UNIQUE NOT NULL,
    city VARCHAR(100) NOT NULL,
    address TEXT,
    school_id UUID REFERENCES schools(id),
    board VARCHAR(20) NOT NULL, -- CBSE, ICSE, STATE, IB
    is_verified BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Children (linked to parent)
CREATE TABLE children (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    name VARCHAR(100),
    gender VARCHAR(10),
    current_class INT NOT NULL, -- 1-12
```

```sql
    school_id UUID REFERENCES schools(id),
    created_at TIMESTAMP DEFAULT NOW()
);

-- Schools (master data)
CREATE TABLE schools (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    city VARCHAR(100) NOT NULL,
    board VARCHAR(20) NOT NULL,
    address TEXT,
    is_verified BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Book Listings
CREATE TABLE listings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    listing_type VARCHAR(20) NOT NULL, -- 'SET' or 'INDIVIDUAL'
    board VARCHAR(20) NOT NULL,
    class INT NOT NULL,
    school_id UUID REFERENCES schools(id),
    city VARCHAR(100) NOT NULL,
    year_of_purchase INT,
    condition VARCHAR(30) NOT NULL, -- UNUSED, ALMOST_NEW,
        WATER_MARKS, UNDERLINED
    exchange_preference VARCHAR(30)[], -- PICKUP, SCHOOL, PORTER
    status VARCHAR(20) DEFAULT 'ACTIVE', -- ACTIVE, RESERVED,
        EXCHANGED, CANCELLED
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Book Items (for individual books in a listing)
CREATE TABLE listing_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    listing_id UUID REFERENCES listings(id) ON DELETE CASCADE,
    subject VARCHAR(100) NOT NULL,
    title VARCHAR(255),
    publisher VARCHAR(100),
    condition VARCHAR(30),
    created_at TIMESTAMP DEFAULT NOW()
);

-- Listing Images
CREATE TABLE listing_images (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    listing_id UUID REFERENCES listings(id) ON DELETE CASCADE,
    listing_item_id UUID REFERENCES listing_items(id),
    image_type VARCHAR(20), -- FRONT, BACK, MIDDLE
    image_url TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Book Requests
```

```sql
CREATE TABLE requests (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    board VARCHAR(20) NOT NULL,
    class INT NOT NULL,
    subjects VARCHAR(100)[], -- optional filter
    school_id UUID REFERENCES schools(id), -- optional
    city VARCHAR(100) NOT NULL,
    min_condition VARCHAR(30), -- minimum acceptable
    status VARCHAR(20) DEFAULT 'OPEN', -- OPEN, MATCHED, FULFILLED,
        CANCELLED
    is_floated BOOLEAN DEFAULT FALSE, -- waiting for future listings
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Matches (when request matches listing)
CREATE TABLE matches (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    listing_id UUID REFERENCES listings(id),
    request_id UUID REFERENCES requests(id),
    giver_id UUID REFERENCES users(id),
    receiver_id UUID REFERENCES users(id),
    status VARCHAR(20) DEFAULT 'PENDING', -- PENDING, ACCEPTED,
        REJECTED, COMPLETED
    exchange_method VARCHAR(30),
    exchange_date DATE,
    exchange_location TEXT,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Notifications Log
CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    type VARCHAR(30) NOT NULL, -- MATCH_FOUND, REQUEST_RECEIVED,
        EXCHANGE_CONFIRMED
    channel VARCHAR(20) NOT NULL, -- PUSH, WHATSAPP, EMAIL
    title VARCHAR(255),
    body TEXT,
    data JSONB,
    is_read BOOLEAN DEFAULT FALSE,
    sent_at TIMESTAMP DEFAULT NOW()
);

-- OTP Storage (use Redis in production)
CREATE TABLE otps (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) NOT NULL,
    otp VARCHAR(6) NOT NULL,
    expires_at TIMESTAMP NOT NULL,
    is_used BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT NOW()
);
```

```sql
-- Sample Papers (Phase 2)
CREATE TABLE sample_papers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    uploaded_by UUID REFERENCES users(id),
    board VARCHAR(20) NOT NULL,
    class INT NOT NULL,
    subject VARCHAR(100) NOT NULL,
    year INT,
    school_id UUID REFERENCES schools(id),
    file_url TEXT NOT NULL,
    downloads INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW()
);
```

3.2 Indexes

```sql
-- Fast lookups for matching
CREATE INDEX idx_listings_search ON listings(board, class, city,
        status);
CREATE INDEX idx_listings_school ON listings(school_id, status);
CREATE INDEX idx_requests_search ON requests(board, class, city,
        status);
CREATE INDEX idx_children_class ON children(user_id, current_class);
```

---

4. API Endpoints (REST)

4.1 Authentication

Method	Endpoint	Description
POST	`/auth/send-otp`	Send OTP to email
POST	`/auth/verify-otp`	Verify OTP, return JWT
POST	`/auth/refresh`	Refresh access token

4.2 Users

Method	Endpoint	Description
POST	`/users/register`	Create new user
GET	`/users/me`	Get current user profile
PUT	`/users/me`	Update profile
POST	`/users/me/children`	Add child
PUT	`/users/me/children/:id`	Update child
DELETE	`/users/me/children/:id`	Remove child

4.3 Listings

Method	Endpoint	Description
POST	`/listings`	Create new listing

Method	Endpoint	Description
GET	`/listings`	Search listings (with filters)
GET	`/listings/:id`	Get listing details
PUT	`/listings/:id`	Update listing
DELETE	`/listings/:id`	Cancel listing
POST	`/listings/:id/images`	Upload images
GET	`/listings/my`	Get my listings

4.4 Requests

Method	Endpoint	Description
POST	`/requests`	Create book request
GET	`/requests`	Get my requests
GET	`/requests/:id`	Get request details
PUT	`/requests/:id`	Update request
DELETE	`/requests/:id`	Cancel request
POST	`/requests/:id/float`	Float request for future match

4.5 Matches & Exchanges

Method	Endpoint	Description
GET	`/matches`	Get my matches
GET	`/matches/:id`	Get match details
POST	`/matches/:id/accept`	Accept match
POST	`/matches/:id/reject`	Reject match
POST	`/matches/:id/schedule`	Schedule exchange
POST	`/matches/:id/complete`	Mark as completed

4.6 Schools (Master Data)

Method	Endpoint	Description
GET	`/schools`	Search schools
GET	`/schools/:id`	Get school details
POST	`/schools/suggest`	Suggest new school

---

5. Matching Algorithm

```
// Pseudo-code for matching engine

async function findMatches(request) {
  // Priority 1: Same school
  let matches = await db.listings.find({
    board: request.board,
    class: request.class,
```

```
    school_id: request.school_id,
    status: 'ACTIVE',
    condition: { $gte: request.min_condition }
  });

  if (matches.length > 0) return matches;

  // Priority 2: Same city + board
  matches = await db.listings.find({
    board: request.board,
    class: request.class,
    city: request.city,
    status: 'ACTIVE',
    condition: { $gte: request.min_condition }
  });

  return matches;
}

// Condition ranking for comparison
const CONDITION_RANK = {
  'UNUSED': 4,
  'ALMOST_NEW': 3,
  'WATER_MARKS': 2,
  'UNDERLINED': 1
};
```

---

6. Notification Flow

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│ New Listing  │ ──▶ │  Check       │ ──▶ │  Send        │
│  Created     │    │ Floated      │    │ WhatsApp     │
└──────────────┘    │ Requests     │    │ + Push       │
                    └──────────────┘    └──────────────┘

┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│ New Request  │ ──▶ │  Search      │ ──▶ │  Show        │
│  Created     │    │ Existing     │    │ Results      │
└──────────────┘    │ Listings     │    │ or Float     │
                    └──────────────┘    └──────────────┘
```

WhatsApp Message Template (Example)

```
📚 BookSwap Alert!

Books matching your request are now available:

📖 Class 8 - CBSE (Full Set)
🏫 Same school: DPS RK Puram
📍 New Delhi
⭐ Condition: Almost New
```

```
Open app to view & request:
[BUTTON: View Books]
```

---

7. Security Considerations

Concern	Solution
Auth	JWT with short expiry, refresh tokens
OTP Brute Force	Rate limit (5 attempts/15 min)
Image Upload	Validate file type, size limit (5MB)
Personal Data	Share only after mutual acceptance
Phone Privacy	Mask phone numbers in listings
SQL Injection	Parameterized queries, ORM

---

8. MVP Development Phases

Phase 1: Core (Weeks 1-4)

- [ ] User auth (email OTP)
- [ ] Profile management
- [ ] School master data
- [ ] Basic listings CRUD

Phase 2: Exchange (Weeks 5-6)

- [ ] Book requests
- [ ] Matching engine
- [ ] Exchange workflow

Phase 3: Notifications (Weeks 7-8)

- [ ] Push notifications (FCM)
- [ ] WhatsApp integration
- [ ] Float request alerts

Phase 4: Polish (Weeks 9-10)

- [ ] Image upload/gallery
- [ ] Search improvements
- [ ] Testing & bug fixes

Phase 5: Launch (Weeks 11-12)

- [ ] Beta testing (1-2 schools)
- [ ] Play Store submission
- [ ] Soft launch

---

9. Hosting & DevOps (MVP)

```
┌──────────────────────────────────────────────────────────────┐
│                        Railway / Render                      │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐    │
│   │  Node.js     │   │  PostgreSQL  │   │    Redis     │    │
│   │  Backend     │   │  (Managed)   │   │  (Managed)   │    │
│   └──────────────┘   └──────────────┘   └──────────────┘    │
└──────────────────────────────────────────────────────────────┘
                              │
                              │
┌──────────────────────────────────────────────────────────────┐
│                       External Services                      │
│                                                              │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐    │
│   │  Cloudinary  │   │   Gupshup    │   │   Firebase   │    │
│   │  (Images)    │   │  (WhatsApp)  │   │    (FCM)     │    │
│   └──────────────┘   └──────────────┘   └──────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

Estimated Costs: | Service | Monthly Cost | |— — —|— — — —–| | Railway (Backend + DB) | ₹1,500 | | Cloudinary (Images) | Free tier | | Firebase (Push) | Free tier | | Gupshup (WhatsApp) | ₹2-3 per message | | Domain | ₹500/year | | **Total** | **~₹5,000/month** |

Next steps: Wireframes & UI Design